

Contents

Watsu Gift Voucher System — Project Documentation	2
Table of Contents	2
1. Project Overview & Purpose	3
What the System Does	3
Real-World Problem	3
Main User Flow	3
Key Features	3
2. High-Level Architecture & Tech Stack	3
Architecture Diagram	3
What Runs Where	4
Full Tech Stack	4
3. Setup & Installation	5
Prerequisites	5
Local Development Setup	5
Required Asset Files	6
Auto-Generated on First Run	6
4. Configuration & Environment	6
Config Files	6
Database URL	6
WebAuthn Configuration	7
Session Configuration	7
5. Module / Code-Level Documentation	7
main.py — Application Entry Point	7
app/database.py — Database Setup	7
app/models.py — ORM Models	8
app/schemas.py — Pydantic Validation Schemas	9
app/rules.py — Business Rules Engine	9
app/qr_module.py — QR Code Generation	10
app/pdf_module.py — PDF Generation	11
app/auth.py — Authentication Module	12
app/utils.py — Utilities	12
6. HTML Templates & UI	13
templates/base.html — Base Layout	13
templates/all_vouchers.html — Main Screen	13
templates/home.html — Dashboard	13
templates/new_voucher.html — New Voucher (Full Page)	13
templates/holder.html — Orderer Detail	13
templates/voucher_view.html — Voucher Detail	13
templates/orderers.html — All Orderers	14
templates/report.html — Filterable Report	14
templates/login.html — Login Page	14
templates/setup.html — First-Time Setup	14
templates/voucher_pdf.html — PDF Template (Reference Only)	14
7. Data & Storage	14
Database Schema	14
File Storage	15

Data Flow	15
8. API Reference	15
JSON API Endpoints	15
HTML Form Endpoints (POST-Redirect-GET)	16
Auth Endpoints	16
Interactive API Docs	17
9. Testing	17
Test Suite	17
Running Tests	17
Test Files	17
Manual Testing	18
10. Deployment & Operations	18
Production Server	18
Deploy Workflow	18
Backup	18
Logs	19
Common Operations	19
11. Security, Performance & Limitations	19
Security	19
Performance	19
Known Limitations	19
12. Known Issues, Roadmap & Future Work	20
Current Issues	20
Roadmap	20
Planned Improvements	20
13. Glossary & External References	20
Key Terms	20
External References	21

Watsu Gift Voucher System — Project Documentation

Version: V1 (Local Python app, manual accounting, no external APIs) **Generated:** 2026-03-21

Repository: [kobysheftel/watsu-voucher](https://github.com/kobysheftel/watsu-voucher) **Production URL:** <https://vouchers.soulwaves.org>

Table of Contents

1. Project Overview & Purpose
2. High-Level Architecture & Tech Stack
3. Setup & Installation
4. Configuration & Environment
5. Module / Code-Level Documentation
6. HTML Templates & UI
7. Data & Storage
8. API Reference
9. Testing
10. Deployment & Operations

11. Security, Performance & Limitations
 12. Known Issues, Roadmap & Future Work
 13. Glossary & External References
-

1. Project Overview & Purpose

What the System Does

The Watsu Gift Voucher System is a web application for creating, managing, sending, and tracking gift vouchers for **גלים ונפש (Waves & Soul)** — a Watsu (water Shiatsu) therapy business in Israel.

Real-World Problem

The business owner (Avigal) needs to: - Create gift vouchers for customers (orderers) who buy them for others - Send vouchers via WhatsApp, email, or print - Track which vouchers have been sent, redeemed, and by whom - Generate PDF voucher documents with Hebrew text, QR codes, and branding - Manage basic accounting (receipt numbers, dates)

Main User Flow

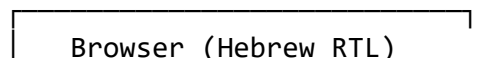
1. Admin logs in (PIN or biometric)
2. Creates an orderer (by mobile number)
3. Creates a draft voucher for that orderer
4. Sends the voucher (via WA/email/print) → cements holder data, generates QR + PDF
5. Can resend the voucher at any time
6. Marks voucher as "used" when redeemed → fully frozen

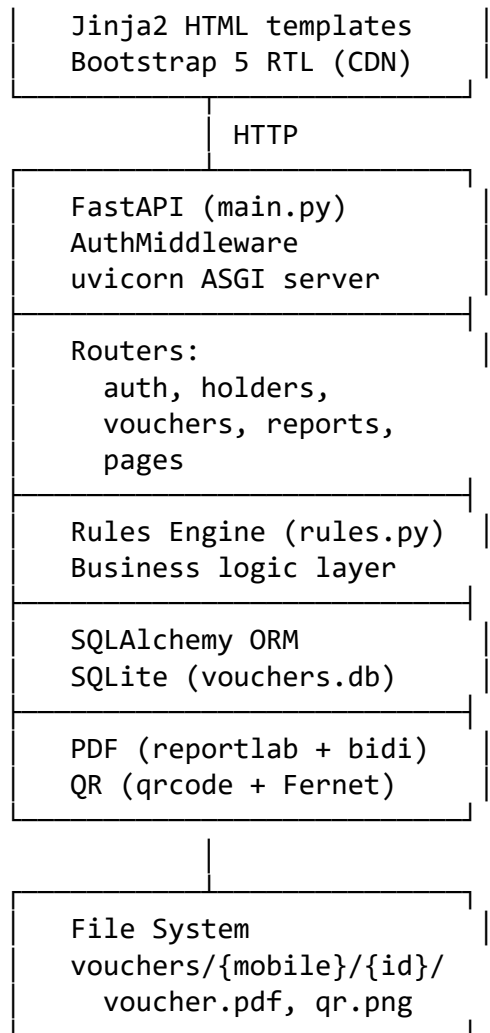
Key Features

- **Orderer management:** Create and edit orderers identified by Israeli mobile numbers
 - **Voucher lifecycle:** draft → sent → used, with strict business rules
 - **PDF generation:** Beautiful A5 water-blue gradient voucher with Hebrew text, pool photo, QR code
 - **QR codes:** Fernet-encrypted payload for future scanning/redemption
 - **WhatsApp sharing:** Native share API on mobile, wa.me fallback on desktop
 - **Biometric auth:** WebAuthn + PIN login with 15-minute inactivity lock
 - **PWA:** Installable on smartphones (manifest + service worker)
 - **Reports:** Filterable report with CSV export (UTF-8 BOM for Excel Hebrew support)
 - **RTL Hebrew UI:** Bootstrap 5 RTL with full Hebrew interface
-

2. High-Level Architecture & Tech Stack

Architecture Diagram





What Runs Where

Component	Local Dev	Production (Hetzner)
Python	3.14	3.12 (venv)
Web server	uvicorn direct	nginx → uvicorn:8100
Database	SQLite file	SQLite file
SSL	None (HTTP)	certbot (HTTPS)
Service	manual	systemd

Full Tech Stack

Discipline	Item	Version
BACKEND	FastAPI	>= 0.115.6
BACKEND	uvicorn	>= 0.32.1
DATABASE	SQLite	via SQLAlchemy >= 2.0.36

Discipline	Item	Version
FRONTEND	Jinja2	>= 3.1.4
FRONTEND	Bootstrap 5 RTL	5.3.2 (CDN)
LIBS	reportlab	>= 4.0
LIBS	python-bidi	>= 0.6.0
LIBS	qrcode	>= 8.0
LIBS	Pillow	>= 11.1.0
LIBS	cryptography	>= 43.0.3
LIBS	webauthn (py_webauthn)	>= 2.0.0
LIBS	aiofiles	>= 24.1.0
LIBS	python-multipart	>= 0.0.12
AUTH	WebAuthn + PIN	15-min inactivity timeout
PWA	manifest + service worker	network-first caching
LANGUAGE	Python	3.12 (server), 3.14 (local)
DEVOPS	Git + GitHub	kobysheftel/watsu-voucher
DEVOPS	systemd + nginx	Hetzner Ubuntu
DEVOPS	certbot	SSL auto-renew

3. Setup & Installation

Prerequisites

- Python 3.12+ installed
- Git
- pip (Python package manager)

Local Development Setup

1. Clone the repository

```
git clone https://github.com/kobysheftel/watsu-voucher.git
cd watsu-voucher
```

2. Create and activate virtual environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Linux/Mac:

```
source venv/bin/activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Run the development server

```
uvicorn main:app --reload --port 8000
```

```
# 5. Open browser
# http://localhost:8000
# First run: you'll be redirected to /auth/setup to create a PIN
```

Required Asset Files

The following files must exist in assets/:

File	Purpose
Pool.png	Pool photo embedded in voucher PDF
voucher-bg.png	Background reference image
fonts/Heebo-Variable.ttf	Hebrew font for PDF generation

Auto-Generated on First Run

File	Location	Purpose
secret.key	config/	Fernet encryption key for QR codes
auth.json	config/	PIN hash + WebAuthn credentials
vouchers.db	project root	SQLite database

4. Configuration & Environment

Config Files

File	Location	Git-tracked	Purpose
config/secretkey	config/	NO (.gitignore)	Fernet encryption key — auto-generated
config/auth.json	config/	NO (.gitignore)	PIN hash, WebAuthn credentials, sessions
vouchers.db	project root	NO (.gitignore)	SQLite database
requirements.txt	project root	YES	Python dependencies

Database URL

Hardcoded in app/database.py:

```
DATABASE_URL = "sqlite:///./vouchers.db"
```

WebAuthn Configuration

Hardcoded in `app/routers/auth.py`:

```
RP_ID = "vouchers.soulwaves.org"
RP_NAME = "Watsu Vouchers"
ORIGIN = "https://vouchers.soulwaves.org"
```

Note: WebAuthn only works on the production domain (HTTPS required). For local development, use PIN login only.

Session Configuration

In `app/auth.py`:

```
SESSION_MAX_AGE = 15 * 60 # 15 minutes inactivity timeout
SESSION_COOKIE_NAME = "voucher_session"
```

Sessions are stored in-memory (single-user, single-process design). Server restart clears all sessions.

5. Module / Code-Level Documentation

`main.py` — Application Entry Point

Purpose: FastAPI app factory with middleware, router registration, and static file mounts.

Key components: - AuthMiddleware — redirects unauthenticated requests to `/auth/login` - Public paths: `/auth/*`, `/static/*`, `/assets/*`, `/docs`, `/openapi.json` - Root / redirects to `/all-vouchers/view` - Static file mounts: `/static`, `/assets`, `/files` (voucher PDFs/QR images)

Run command:

```
uvicorn main:app --reload --port 8000
```

`app/database.py` — Database Setup

Purpose: SQLAlchemy engine, session factory, and DB initialization.

Key functions:

Function	Parameters	Returns	Description
<code>init_db()</code>	None	None	Creates all tables via <code>Base.metadata.create_all()</code> . Called on app startup.
<code>get_db()</code>	None	Session (generator)	FastAPI dependency — yields a DB session, closes when done.

app/models.py — ORM Models

Purpose: SQLAlchemy table definitions for the three core tables.

Holder — Orderer/Customer

Column	Type	Description
mobile	Text (PK)	Israeli mobile 05X-XXXXXXX (10 digits)
name	Text	Full name
email	Text (nullable)	Email address
created_at	DateTime	Creation timestamp
updated_at	DateTime	Last update timestamp

Relationship: vouchers — all vouchers linked to this holder (via mobile text match).

Editability rule: Editable while all vouchers are draft/sent. Locked if any voucher is used.

Voucher — Gift Voucher

Column	Type	Description
voucher_id	Text (PK)	Format: {mobile}-{sequence:03d} e.g. 0541234567-001
mobile	Text	Reference to holder's mobile (not a live FK)
sequence	Integer	Position in holder's voucher sequence
voucher_type	Text	"0000" (single) or "0000" (couple)
valid_until	Date	Expiry date
issued_at	DateTime	Creation timestamp
first_sent_at	DateTime (nullable)	Set once on first send, never changes
status	Text	"draft" / "sent" / "used"
display_name	Text (nullable)	Custom name on PDF (if empty, shows generic text)
receipt_number	Text (nullable)	Manual accounting field
receipt_date	Date (nullable)	Manual accounting field
notes	Text (nullable)	Free-text notes
qr_path	Text (nullable)	Path to QR PNG file (set on first send)
pdf_path	Text (nullable)	Path to PDF file (updated every send/resend)
holder_name	Text (nullable)	Cemented snapshot — copied from Holder on first send
holder_mobile	Text (nullable)	Cemented snapshot
holder_email	Text (nullable)	Cemented snapshot

Properties: - is_frozen — True when status == "used" - is_cemented — True when first_sent_at is not None

Sending — Send/Resend Event Log

Column	Type	Description
id	Integer (PK)	Auto-increment
voucher_id	Text (FK)	References vouchers.voucher_id
sent_at	DateTime	Timestamp of send event
sent_via	Text	"WA" / "EMAIL" / "PRINT"
note	Text (nullable)	Optional note about the send

Never updated — only inserted. Each send/resend creates a new row.

Display labels: | Value | Display | |——-|——-| | WA | ׁ ׁ ׁ ׁ ׁ | | EMAIL | ׁ ׁ ׁ | | PRINT | ׁ ׁ ׁ |

app/schemas.py — Pydantic Validation Schemas

Purpose: Request/response validation for all API endpoints.

Key schemas:

Schema	Type	Description
HolderCreate	Request	mobile (validated Israeli format), name, email
HolderUpdate	Request	name, email (optional)
HolderOut	Response	Full holder data + can_edit flag
VoucherCreate	Request	mobile, voucher_type, (יחיד/זוגי) valid_until (not in past)
VoucherUpdate	Request	receipt_number, receipt_date, notes
VoucherOut	Response	Full voucher + cemented data + computed flags + sendings list
VoucherListItem	Response	Compact row for table views
SendRequest	Request	sent_via (WA/EMAIL/PRINT), note
SendingOut	Response	Single sending event with display label
SearchResult	Response	Flat row for search/report results

Mobile validation: Strips separators, validates 05XXXXXXXX (10 digits).

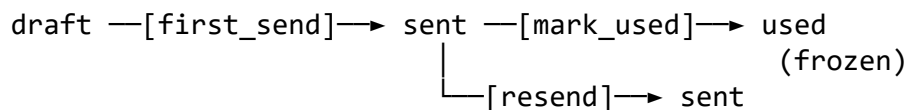
app/rules.py — Business Rules Engine

Purpose: Single source of truth for all business logic. HTTP-agnostic — receives a DB session, returns ORM objects.

Key functions:

Function	Parameters	Returns	Description
<code>can_edit_holder(db, mobile)</code>	<code>Session, str</code>	<code>bool</code>	True if no vouchers are "used"
<code>get_or_create_holder(db, mobile, name, email)</code>	<code>Session, str, str, str None</code>	<code>(Holder, bool)</code>	Creates or retrieves holder
<code>update_holder(db, mobile, name, email)</code>	<code>Session, str, str None, str None</code>	<code>Holder</code>	Updates holder; raises <code>RulesError</code> if locked
<code>next_sequence(db, mobile)</code>	<code>Session, str</code>	<code>int</code>	Next sequence number for a mobile
<code>create_voucher(db, mobile, type, valid_until, ...)</code>	<code>Session, ...</code>	<code>Voucher</code>	Creates draft voucher
<code>first_send(db, voucher_id, sent_via, note)</code>	<code>Session, str, str, str None</code>	<code>Voucher</code>	Full first-send sequence (cement + QR + PDF + log)
<code>resend(db, voucher_id, sent_via, note)</code>	<code>Session, str, str, str None</code>	<code>Voucher</code>	Resend (reuse QR, new PDF, new log entry)
<code>mark_used(db, voucher_id)</code>	<code>Session, str</code>	<code>Voucher</code>	Freeze voucher permanently
<code>update_voucher_fields(db, voucher_id, ...)</code>	<code>Session, str, ...</code>	<code>Voucher</code>	Update receipt/notes (blocked if used)

Status transition rules:



Cementing: On first send, holder details (name, mobile, email) are copied into the voucher's `cemented_*` fields. These never change again, even if the holder is later edited.

Custom exception: `RulesError` — raised for all business rule violations.

app/qr_module.py — QR Code Generation

Purpose: Generate Fernet-encrypted QR codes for voucher verification.

Payload format:

```
{"id": "0541234567-001", "ts": "2026-03-21 12:00:00"}
```

Key functions:

Function	Parameters	Returns	Description
<code>generate_qr(voucher, Path folder)</code>	<code>Voucher, Path</code>	<code>Path</code>	Generates encrypted QR PNG. Called ONCE per voucher.
<code>decrypt_qr(token)</code>	<code>str</code>	<code>dict</code>	Decrypts a QR token back to {id, ts} payload

Encryption: - Fernet symmetric encryption (from cryptography library) - Key auto-generated on first run, saved to `config/secret.key` - QR image: auto-sized, M-level error correction, 6px box size

app/pdf_module.py — PDF Generation

Purpose: Generate A5 voucher PDFs with Hebrew text using reportlab + python-bidi.

Design decision: xhtml2pdf was abandoned because it cannot render Hebrew glyphs. Reportlab with python-bidi handles RTL text correctly.

Layout (top to bottom): 1. Water-blue gradient background (3-color blend) 2. header ונפש "גלים in teal (30pt) 3. subtitle "וואטסו" טיפולי (14pt) 4. Wavy decorative line 5. Pool photo (rounded, centered, with white frame) 6. Personalized or anonymous title: - With `display_name`: כיף "איזה / {name} / {type}" - Without: כיף "איזה" מתנה שובר קיבלת / חתנה שובר קיבלת / חתנה שובר קיבלת / חתנה שובר קיבלת 7. Ornamental divider (lines + diamond) 8. Treatment description (3 lines) 9. Contact info: "אביגל 050-4014696" 10. Expiry date 11. QR code (small, centered) 12. Voucher ID + receipt number (bottom, 6pt)

Key function:

Function	Parameters	Returns	Description
<code>generate_pdf(voucher, Path folder)</code>	<code>Voucher, Path</code>	<code>Path</code>	Generates A5 PDF, returns saved path

Font: Heebo (Hebrew variable font) from `assets/fonts/Heebo-Variable.ttf`. Registered once per process.

Color palette:

Name	Hex	Usage
TEAL	#2a7f8e	Main title
TEAL_DARK	#1a5f6e	Subtitles, emphasis
WARM	#5a4a3a	Body text
GREY	#666666	Secondary text
LGREY	#999999	Footer text
BG_TOP	#d4eef2	Gradient top
BG_MID	#e8f4f0	Gradient middle

Name	Hex	Usage
BG_BOT	#dce8e4	Gradient bottom
ORNAMENT	#7ab5b0	Decorative elements

app/auth.py — Authentication Module

Purpose: PIN + WebAuthn biometric authentication for single-user admin access.

Key functions:

Function	Parameters	Returns	Description
hash_pin(pin)	str	dict	Creates salted SHA-256 hash
verify_pin(pin, pin_data)	str, dict	bool	Verifies PIN against stored hash
is_setup_complete()	None	bool	True if PIN has been configured
save_pin(pin)	str	None	Hash and store PIN to auth.json
check_pin(pin)	str	bool	Verify PIN against stored credentials
get_webauthn_credentials()	None	list	Get stored WebAuthn credentials
save_webauthn_credential(credential)	credential	None	Add a WebAuthn credential
create_session(response)	Response	str	Create session + set cookie
is_authenticated(response)	Response	bool	Check session validity (sliding window)
logout(response)	Response	None	Clear session

Session design: - In-memory dict (_sessions: dict[str, float]) - Sliding window: timeout resets on every authenticated request - 15-minute inactivity timeout - Cookie: voucher_session, httponly, samesite=lax

app/utils.py — Utilities

Purpose: Shared utility functions.

Function	Returns	Description
utcnow()	datetime	Current UTC time as naive datetime (SQLite-compatible)

6. HTML Templates & UI

All templates use Jinja2 with Bootstrap 5 RTL (CDN). The UI is fully in Hebrew (RTL).

`templates/base.html` — Base Layout

- Bootstrap 5.3.2 RTL (CDN)
- PWA manifest link + service worker registration
- Inactivity lock (15-minute JS timer, redirects to `/auth/logout`)
- Top navbar with navigation and logout button
- Flash message display (from URL params `?flash=...&flash_type=...`)
- POST-Redirect-GET pattern throughout

`templates/all_vouchers.html` — Main Screen

- All vouchers table with sortable/filterable columns
- Excel-like column filters (status, type, orderer)
- "שובר" "חדש" (new voucher) button
- Inline status badges (color-coded)
- Links to voucher detail pages

`templates/home.html` — Dashboard

- Stats cards (total, draft, sent, used, orderers)
- Orderer search form (by name, phone, or email)
- Search results list with links to orderer pages

`templates/new_voucher.html` — New Voucher (Full Page)

- Smart orderer search: phone lookup, pick from list, or create new
- Voucher form: type (יחיד/זוגי), expiry (default: 6 months), display name, receipt number, receipt date, notes
- Creates orderer if needed + creates draft voucher in one flow

`templates/holder.html` — Orderer Detail

- Orderer info card (name, mobile, email) with edit capability
- All vouchers for this orderer in a table
- Inline actions per voucher
- New voucher modal for this orderer

`templates/voucher_view.html` — Voucher Detail

- Full voucher info: ID, status, type, dates, cemented holder data
- Action buttons (context-sensitive based on status):
 - Draft: Send (WA/Email/Print)
 - Sent: Resend, Mark Used, View PDF
 - Used: View PDF (read-only)
- WhatsApp sharing: `navigator.share()` on mobile, `wa.me` link on desktop

- Sending history table
- Receipt/notes edit form (blocked when used)
- QR code display

templates/orderers.html — All Orderers

- List of all orderers with voucher counts
- Links to individual orderer pages

templates/report.html — Filterable Report

- Filter by: status, type, date range, receipt number
- Results table
- CSV export button (UTF-8 BOM for Excel Hebrew)

templates/login.html — Login Page

- Biometric login button (WebAuthn, if credential registered)
- PIN fallback (4-6 digits)
- Error messages

templates/setup.html — First-Time Setup

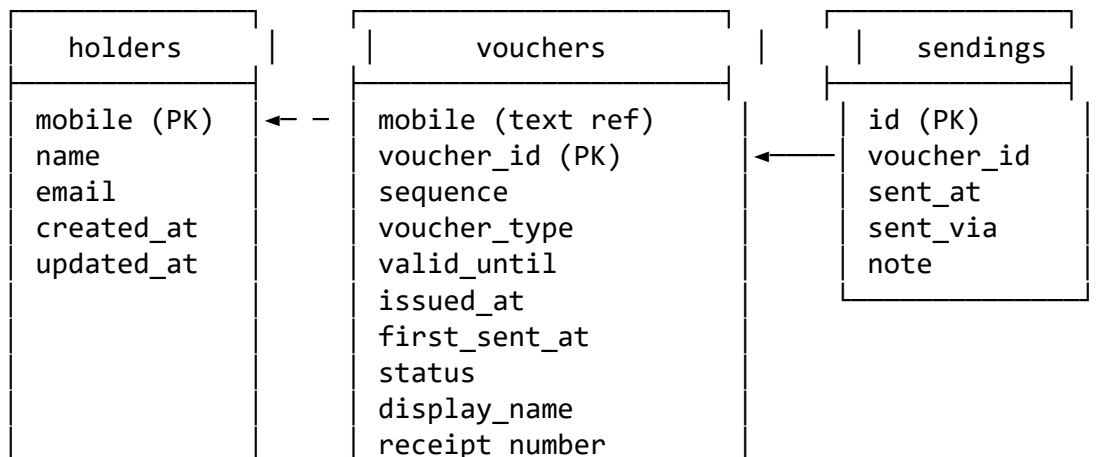
- Step 1: PIN creation (4-6 digits with confirmation)
- Step 2: Biometric registration (optional WebAuthn)

templates/voucher_pdf.html — PDF Template (Reference Only)

Reference HTML template — not actually used (reportlab generates PDFs directly).

7. Data & Storage

Database Schema



	receipt_date
	notes
	qr_path
	pdf_path
	holder_name (cemented)
	holder_mobile (cem.)
	holder_email (cem.)

Design note: vouchers.mobile is a plain text reference to holders.mobile, NOT a live foreign key. This ensures voucher history survives even if holder deletion is ever added.

File Storage

```
vouchers/
  {mobile}/
    {voucher_id}/
      voucher.pdf      # Generated on every send/resend
      qr.png           # Generated ONCE on first send, reused forever
```

Data Flow

```
Browser form → POST → FastAPI router → rules.py → SQLAlchemy → SQLite
                                     |
                                     ├── qr_module.py → qr.png
                                     └── pdf_module.py → voucher.pdf
                                     |
                                     response ←
```

8. API Reference

JSON API Endpoints

Holders

Method	Path	Description
POST	/holders	Create or retrieve holder by mobile
GET	/holders/{mobile}	Holder details + all vouchers
PUT	/holders/{mobile}	Update name/email (blocked if has used voucher)

Vouchers

Method	Path	Description
POST	/vouchers	Create new draft voucher
GET	/vouchers	List all vouchers (filterable: status, mobile, type, receipt, dates)
GET	/vouchers/{id}	Full voucher details + sending history
PUT	/vouchers/{id}	Update receipt/notes (blocked if used)
PUT	/vouchers/{id}/send	First send (cement + QR + PDF + log)
PUT	/vouchers/{id}/resend	Resend (reuse QR, new PDF, new log)
PUT	/vouchers/{id}/use	Mark used (freeze)
GET	/vouchers/{id}/pdf	Serve PDF file
GET	/vouchers/{id}/qr	Serve QR image
GET	/vouchers/{id}/sendfull	Full sending history

Reports & Search

Method	Path	Description
GET	/search?q=...	Search by name, mobile, email, voucher ID
GET	/report	Full report with filters; ?fmt=csv for CSV export
GET	/home/data	Dashboard stats (total, draft, sent, used, holders)

HTML Form Endpoints (POST-Redirect-GET)

Method	Path	Description
POST	/form/holders/new	Create holder via form
POST	/form/holders/{mobile}/update	Update holder via form
POST	/form/vouchers/new	Create voucher (holder must exist)
POST	/form/vouchers/newfull	Create orderer + voucher in one step full
POST	/form/vouchers/{id}/resend	Send or resend (auto-detects status)
POST	/form/vouchers/{id}/markused	Mark used
POST	/form/vouchers/{id}/update	Update receipt/notes

Auth Endpoints

Method	Path	Description
GET	/auth/login	Login page
POST	/auth/login/pin	PIN login
GET	/auth/setup	First-time setup page
POST	/auth/setup	Save PIN
GET	/auth/setup/biometric	Biometric registration page

Method	Path	Description
GET	/auth/logout	Logout
POST	/auth/webauthn/register	WebAuthn registration options
POST	/auth/webauthn/register	WebAuthn registration verify
POST	/auth/webauthn/login	WebAuthn login options
POST	/auth/webauthn/login	WebAuthn login verify

Interactive API Docs

FastAPI auto-generates Swagger UI at /docs (accessible without auth).

9. Testing

Test Suite

- **48 tests total** (21 rules engine + 27 API endpoint tests)
- All tests use in-memory SQLite — no files touched on disk
- API tests include auth fixture (auto-creates PIN + session)

Running Tests

All tests

```
python -m pytest tests/ -v
```

Rules engine only

```
python -m pytest tests/test_rules.py -v
```

API endpoints only

```
python -m pytest tests/test_api.py -v
```

Test Files

tests/test_rules.py — Rules Engine Tests (21 tests) Tests business logic directly via rules.py functions:

- Holder CRUD: create, retrieve, update, blocked-when-used
- Voucher creation: sequential IDs, no-holder error
- First send: cementing, logging, draft-only, cement survives holder edit
- Resend: new log row, first_sent_at unchanged, blocked on draft/used
- Mark used: status change, draft-blocked, double-use blocked
- Update fields: receipt update, blocked-when-used

tests/test_api.py — API Endpoint Tests (27 tests) Tests HTTP endpoints via FastAPI Test-Client:

- Holder endpoints: create, retrieve, get-with-vouchers, not-found, update, blocked, validation

- Voucher CRUD: create, no-holder, list, filter, get, not-found, receipt update, blocked
- Send/resend/use: first send, resend, mark used, resend-after-used blocked
- Sending history: multiple sends, display labels
- Search & report: by name, no results, JSON, CSV, home data

Manual Testing

1. Start the server: `uvicorn main:app --reload --port 8000`
 2. Open `http://localhost:8000` in a browser
 3. Complete initial setup (PIN)
 4. Create a holder → create a voucher → send → resend → mark used
 5. Test WhatsApp sharing from a mobile device connected to the same network
-

10. Deployment & Operations

Production Server

- **Host:** Hetzner Cloud (Ubuntu)
- **Domain:** `vouchers.soulwaves.org`
- **SSH:** `ssh -i ~/.ssh/Hetzner_new root@iceland.soulwaves.org`
- **App path:** `/var/www/sites/vouchers/`
- **Python venv:** `/var/www/sites/vouchers/venv/`
- **Service:** systemd (vouchers)
- **Reverse proxy:** nginx → uvicorn on port 8100
- **SSL:** certbot with auto-renewal

Deploy Workflow

1. Push changes Locally
`git push origin master`

2. SSH to server
`ssh -i ~/.ssh/Hetzner_new root@iceland.soulwaves.org`

3. Pull and restart
`cd /var/www/sites/vouchers`
`git pull`
`systemctl restart vouchers`

4. For DB schema changes: run ALTER TABLE before restart

Backup

Daily backup script: `backup.bat` (Windows scheduled task at 23:00)

Backs up: - vouchers/ directory (PDFs, QR images) → backup/vouchers/ - *.db (SQLite database) → backup/

```
robocopy "%SRC%\vouchers" "%DST%\vouchers" /MIR
robocopy "%SRC%" "%DST%" *.db
```

Logs

- **uvicorn logs:** stdout (captured by systemd journal on production)
- **View server logs:** `journalctl -u vouchers -f`
- **Backup log:** `backup/backup.log`

Common Operations

Task	Command
Start dev server	<code>uvicorn main:app --reload --port 8000</code>
Restart production	<code>systemctl restart vouchers</code>
View production logs	<code>journalctl -u vouchers -f</code>
Check production status	<code>systemctl status vouchers</code>
Run tests	<code>python -m pytest tests/ -v</code>

11. Security, Performance & Limitations

Security

- **Authentication:** PIN (4-6 digits, salted SHA-256) + WebAuthn biometric
- **Session:** HTTP-only cookie, samesite=lax, 15-minute sliding timeout
- **QR encryption:** Fernet symmetric encryption (prevents token forgery)
- **Sensitive files:** `config/`, `vouchers/`, `*.db` excluded from git
- **HTTPS:** enforced on production via certbot
- **Session storage:** in-memory (cleared on server restart)

Performance

- **SQLite:** adequate for single-user, low-volume use case
- **In-memory sessions:** O(1) lookup, no DB overhead
- **PDF generation:** reportlab is fast (~100ms per voucher)
- **Service worker:** network-first strategy with offline fallback
- **CDN:** Bootstrap loaded from CDN (reduces server load)

Known Limitations

- **Single user:** designed for one admin user; no multi-user support
- **In-memory sessions:** lost on server restart (user must re-login)
- **WebAuthn:** requires HTTPS (production only; PIN for local dev)
- **No email sending:** records email as a sending method but doesn't actually send emails
- **SQLite:** not suitable for concurrent writes (adequate for single-user)
- **No API authentication:** JSON API endpoints share session auth (no API keys/tokens)

- **Hardcoded WebAuthn origin:** `vouchers.soulwaves.org` is hardcoded; local dev cannot use biometric

12. Known Issues, Roadmap & Future Work

Current Issues

- No actual email sending (records only)
- WebAuthn configuration hardcoded to production domain
- PDF design may need further polish

Roadmap

Version	Description
V1 (current)	Local Python app, manual accounting, no external APIs
V2	Green Invoice API integration (automated invoicing)
V3	JS/browser version (client-side, no Python backend)

Planned Improvements

- Voucher ID = receipt number integration
- PDF design polish (additional branding, logo/wave graphics)
- QR code scanning for redemption
- Actual email sending capability
- Configuration file for WebAuthn RP_ID/ORIGIN (dev vs. production)

13. Glossary & External References

Key Terms

Term (Hebrew)	English	Description
מזמין	Orderer	Person who buys/orders vouchers
שובר	Voucher	Gift voucher document
צימנט (Cement)	Cement	Copy holder details into voucher on first send — frozen forever
טיוטה (Draft)	Draft	Initial voucher state before sending
נשלח (Sent)	Sent	Voucher has been sent at least once
מומש (Used)	Used/Redeemed	Final state — completely frozen
יחיד	Single	Single-person treatment voucher
זוגי	Couple	Couple treatment voucher
וואטסו	Watsu	Water Shiatsu — aquatic therapy
ונפש גלים	Waves & Soul	Business name

External References

Resource	URL
FastAPI docs	https://fastapi.tiangolo.com/
SQLAlchemy docs	https://docs.sqlalchemy.org/
reportlab user guide	https://docs.reportlab.com/
python-bidi	https://pypi.org/project/python-bidi/
py_webauthn	https://pypi.org/project/webauthn/
Fernet encryption	https://cryptography.io/en/latest/fernet/
Bootstrap 5 RTL	https://getbootstrap.com/docs/5.3/getting-started/rtl/
PWA docs	https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps